Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	**Slot**: L43+L44
Regno: 23BCE1145	**Name**: S Vishwajith

Ex 1 – 1A, 1B and 1C Combined
EX1A – Classical Cryptographic Techniques

Code (Caesar Cipher):

Sender:

```
import java.util.*;

import java.io.*;

import java.net.*;

public class Caesar_Sender {

   private Socket socket = null;

   private DataInputStream input = null;

   private DataOutputStream output = null;

   private Scanner sc = null;

   public Caesar_Sender(String address, int port) {

      try {

         socket = new Socket(address, port);

         System.out.println("Client is connected!");

         input = new DataInputStream(socket.getInputStream());

         output = new DataOutputStream(socket.getOutputStream());

         sc = new Scanner(System.in);

      } catch (UnknownHostException u) {

         System.out.println(u);

      } catch (IOException i) {
```

```
        System.out.println(i);
    }
}
String caesarEncrypt(String pt, int k) {
    StringBuilder ct = new StringBuilder();
    for (char c : pt.toCharArray()) {
        if (Character.isUpperCase(c)) {
            int ch = (int) c - 'A';
            ch = (ch + k) % 26;
            char cs = (char) (ch + 65);
            ct.append(cs);
        } else if (Character.isLowerCase(c)) {
            int ch = (int) c - 'a';
            ch = (ch + k) % 26;
            char cs = (char) (ch + 97);
            ct.append(cs);
        } else {
            ct.append(c);
        }
    }
    return ct.toString();
}
public void Work() {
    try {
        System.out.print("Enter the message: ");
        String[] ips = sc.nextLine().trim().split("\\s+");
        System.out.print("Enter the key: ");
        int k = sc.nextInt();
        Vector<String> ct = new Vector<>();
        for (String word : ips) {
            ct.add(caesarEncrypt(word, k));
```

```
            }
            String cts = String.join(" ", ct);
            System.out.println("The encrypted text is: " + cts);
            output.writeInt(k);
            output.writeUTF(cts);
            System.out.println(input.readUTF());
            input.close();
            output.close();
            sc.close();
        } catch (IOException i) {
            System.out.println(i);
        }
    }
    public static void main(String[] args) {
        Caesar_Sender cc = new Caesar_Sender("127.0.0.1", 5431);
        cc.Work();
    }
}
```

Receiver:

```
import java.util.*;
import java.io.*;
import java.net.*;
public class Caesar_Receiver {
    private ServerSocket server = null;
    private Socket socket = null;
    private DataInputStream input = null;
    private DataOutputStream output = null;
    public Caesar_Receiver(int port) {
        try {
            server = new ServerSocket(port);
            socket = server.accept();
```

```
        System.out.println("Client is connected!\n");
        input = new DataInputStream(socket.getInputStream());
        output = new DataOutputStream(socket.getOutputStream());
    } catch (UnknownHostException u) {
        System.out.println(u);
    } catch (IOException i) {
        System.out.println(i);
    }
}
String caesarDecrypt(String ct, int k) {
    StringBuilder pt = new StringBuilder();
    for (char c : ct.toCharArray()) {
        if (Character.isUpperCase(c)) {
            int ch = (int) c - 'A';
            ch = (ch - k) % 26;
            if (ch < 0) {
                ch += 26;
            }
            char cs = (char) (ch + 65);
            pt.append(cs);
        } else if (Character.isLowerCase(c)) {
            int ch = (int) c - 'a';
            ch = (ch - k) % 26;
            if (ch < 0) {
                ch += 26;
            }
            char cs = (char) (ch + 97);
            pt.append(cs);
        } else {
            pt.append(c);
        }
```

```
        }
        return pt.toString();
    }
    public void Work() {
        try {
            int k = input.readInt();
            String ip = input.readUTF();
            String[] ips = ip.trim().split("\\s+");
            Vector<String> ct = new Vector<>();
            for (String word : ips) {
                ct.add(caesarDecrypt(word, k));
            }
            System.out.println("The received encrypted text is: " + ip);
            System.out.println("The received key is: " + k);
            String cts = String.join(" ", ct);
            System.out.println("The decrypted text is: " + cts);
            output.writeUTF("Success from receiver!");
            input.close();
            output.close();
        } catch (IOException i) {
            System.out.println(i);
        }
    }
    public static void main(String[] args) {
        Caesar_Receiver cs = new Caesar_Receiver(5431);
        cs.Work();
    }
}
```

Output (Caesar Cipher):

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1> java .\Caesar_Sender.java
Client is connected!
Enter the message: Meet me.
Enter the key: 3
The encrypted text is: Phhw ph.
Success from receiver!
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1> java .\Caesar_Receiver.java
Client is connected!

The received encrypted text is: Phhw ph.
The received key is: 3
The decrypted text is: Meet me.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1>
```

Code (Vigenère Cipher):

Sender:

```
import java.util.*;

import java.io.*;

import java.net.*;


public class Vignere_Sender {

    private Socket socket = null;

    private DataInputStream input = null;

    private DataOutputStream output = null;

    private Scanner sc = null;

    int pos = 0;


    public Vignere_Sender(String address, int port) {

        try {

            socket = new Socket(address, port);

            System.out.println("Client is connected!");

            input = new DataInputStream(socket.getInputStream());

            output = new DataOutputStream(socket.getOutputStream());

            sc = new Scanner(System.in);

        } catch (UnknownHostException u) {

            System.out.println(u);

        } catch (IOException i) {

            System.out.println(i);

        }

    }


    String vignereEncrpyt(String pt, String k) {

        StringBuilder ct = new StringBuilder();

        int klen = k.length();
```

```
    for (char c : pt.toCharArray()) {
        char keyChar = k.charAt(pos);
        int shift;
        if (Character.isUpperCase(keyChar)) {
            shift = keyChar - 'A';
        } else {
            shift = keyChar - 'a';
        }

        if (Character.isUpperCase(c)) {
            int ch = (int) c - 'A';
            ch = (ch + shift) % 26;
            char cs = (char) (ch + 65);
            ct.append(cs);
            pos = (pos + 1) % klen;
        } else if (Character.isLowerCase(c)) {
            int ch = (int) c - 'a';
            ch = (ch + shift) % 26;
            char cs = (char) (ch + 97);
            ct.append(cs);
            pos = (pos + 1) % klen;
        } else {
            ct.append(c);
        }
    }

    return ct.toString();
}

public void Work() {
    try {
```

```
            System.out.print("Enter the message: ");
            String[] ips = sc.nextLine().trim().split("\\s+");
            System.out.print("Enter the key: ");
            String k = sc.next().trim();
            Vector<String> ct = new Vector<>();
            for (String word : ips) {
                ct.add(vignereEncrpyt(word, k));
            }
            String cts = String.join(" ", ct);
            System.out.println("The encrypted text is: " + cts);
            output.writeUTF(k);
            output.writeUTF(cts);
            System.out.println(input.readUTF());
            input.close();
            output.close();
            sc.close();
        } catch (IOException i) {
            System.out.println(i);
        }
    }

    public static void main(String[] args) {
        Vignere_Sender vs = new Vignere_Sender("127.0.0.1", 5431);
        vs.Work();
    }
}
```

Receiver:

```
import java.util.*;
import java.io.*;
import java.net.*;
```

```
public class Vignere_Receiver {
    private ServerSocket server = null;
    private Socket socket = null;
    private DataInputStream input = null;
    private DataOutputStream output = null;
    int pos = 0;

    public Vignere_Receiver(int port) {
        try {
            server = new ServerSocket(port);
            socket = server.accept();
            System.out.println("Client is connected!\n");
            input = new DataInputStream(socket.getInputStream());
            output = new DataOutputStream(socket.getOutputStream());
        } catch (UnknownHostException u) {
            System.out.println(u);
        } catch (IOException i) {
            System.out.println(i);
        }
    }

    String vignereDecrypt(String ct, String k) {
        StringBuilder pt = new StringBuilder();
        int klen = k.length();

        for (char c : ct.toCharArray()) {
            char keyChar = k.charAt(pos);
            int shift;
            if (Character.isUpperCase(keyChar)) {
                shift = keyChar - 'A';
            } else {
```

```
            shift = keyChar - 'a';
        }

        if (Character.isUpperCase(c)) {
            int ch = (int) c - 'A';
            ch = (ch - shift) % 26;
            if (ch < 0) {
                ch += 26;
            }
            char vr = (char) (ch + 65);
            pt.append(vr);
            pos = (pos + 1) % klen;
        } else if (Character.isLowerCase(c)) {
            int ch = (int) c - 'a';
            ch = (ch - shift) % 26;
            if (ch < 0) {
                ch += 26;
            }
            char vr = (char) (ch + 97);
            pt.append(vr);
            pos = (pos + 1) % klen;
        } else {
            pt.append(c);
        }
    }

    return pt.toString();
}

public void Work() {
    try {
```

```
            String k = input.readUTF();
            String ip = input.readUTF();
            String[] ips = ip.trim().split("\\s+");
            Vector<String> ct = new Vector<>();
            for (String word : ips) {
                ct.add(vignereDecrypt(word, k));
            }
            System.out.println("The received encrypted text is: " + ip);
            System.out.println("The received key is: " + k);
            String cts = String.join(" ", ct);
            System.out.println("The decrypted text is: " + cts);
            output.writeUTF("Success from receiver!");
            input.close();
            output.close();
        } catch (IOException i) {
            System.out.println(i);
        }
    }

    public static void main(String[] args) {
        Vignere_Receiver vr = new Vignere_Receiver(5431);
        vr.Work();
    }
}
```

Output (Vigenère Cipher):

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1\Vignere Cipher> java .\Vignere_Sender.java
Client is connected!
Enter the message: Geeks for Geeks.
Enter the key: Ayush
The encrypted text is: Gcycz fml Yleim.
Success from receiver!
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1\Vignere Cipher>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1\Vignere Cipher> java .\Vignere_Receiver.java
Client is connected!

The received encrypted text is: Gcycz fml Yleim.
The received key is: Ayush
The decrypted text is: Geeks for Geeks.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1\Vignere Cipher>
```

Code (Vernam Cipher):

Sender:

```
import java.util.*;

import java.io.*;

import java.net.*;


public class Vernam_Sender {

    private Socket socket = null;

    private DataInputStream input = null;

    private DataOutputStream output = null;

    private Scanner sc = null;

    int pos = 0;


    public Vernam_Sender(String address, int port) {

        try {

            socket = new Socket(address, port);

            System.out.println("Client is connected!");

            input = new DataInputStream(socket.getInputStream());

            output = new DataOutputStream(socket.getOutputStream());

            sc = new Scanner(System.in);

        } catch (UnknownHostException u) {

            System.out.println(u);

        } catch (IOException i) {

            System.out.println(i);

        }

    }


    String vernamEncrypt(String pt, String k) {

        StringBuilder ct = new StringBuilder();

        for (int i = 0; i < pt.length(); i++) {

            char pChar = pt.charAt(i);
```

```
        char kChar = k.charAt(i);
        int pc, kc;
        if (Character.isUpperCase(pChar)) {
            pc = (int) pChar - 'A';
        } else {
            pc = (int) pChar - 'a';
        }
        if (Character.isUpperCase(kChar)) {
            kc = (int) kChar - 'A';
        } else {
            kc = (int) kChar - 'a';
        }
        int cc = (pc ^ kc);
        if (Character.isUpperCase(pChar)) {
            char cChar = (char) (cc + 'A');
            ct.append(cChar);
        } else {
            char cChar = (char) (cc + 'a');
            ct.append(cChar);
        }
    }
    return ct.toString();
}

public void Work() {
    try {
        System.out.print("Enter the message: ");
        String ips = sc.next().trim();
        System.out.print("Enter the key: ");
        String k = sc.next().trim();
        String cts = vernamEncrypt(ips, k);
```

```
            System.out.println("The encrypted text is: " + cts);
            output.writeUTF(k);
            output.writeUTF(cts);
            System.out.println(input.readUTF());
            input.close();
            output.close();
            sc.close();
        } catch (IOException i) {
            System.out.println(i);
        }
    }

    public static void main(String[] args) {
        Vernam_Sender vs = new Vernam_Sender("127.0.0.1", 5431);
        vs.Work();
    }
}
```

Receiver:

```
import java.io.*;
import java.net.*;

public class Vernam_Receiver {
    private ServerSocket server = null;
    private Socket socket = null;
    private DataInputStream input = null;
    private DataOutputStream output = null;
    int pos = 0;

    public Vernam_Receiver(int port) {
        try {
            server = new ServerSocket(port);
```

```
        socket = server.accept();
        System.out.println("Client is connected!\n");
        input = new DataInputStream(socket.getInputStream());
        output = new DataOutputStream(socket.getOutputStream());
    } catch (UnknownHostException u) {
        System.out.println(u);
    } catch (IOException i) {
        System.out.println(i);
    }
}

String vernamDecrypt(String ct, String k) {
    StringBuilder pt = new StringBuilder();
    for (int i = 0; i < ct.length(); i++) {
        char cChar = ct.charAt(i);
        char kChar = k.charAt(i);
        int cc, kc;
        if (Character.isUpperCase(cChar)) {
            cc = (int) cChar - 'A';
        } else {
            cc = (int) cChar - 'a';
        }
        if (Character.isUpperCase(kChar)) {
            kc = (int) kChar - 'A';
        } else {
            kc = (int) kChar - 'a';
        }
        int pc = (cc ^ kc);
        if (Character.isUpperCase(cChar)) {
            char pChar = (char) (pc + 'A');
            pt.append(pChar);
```

```
            } else {
                char pChar = (char) (pc + 'a');
                pt.append(pChar);
            }
        }
        return pt.toString();
    }

    public void Work() {
        try {
            String k = input.readUTF();
            String ip = input.readUTF();
            String cts = vernamDecrypt(ip, k);
            System.out.println("The received encrypted text is: " + ip);
            System.out.println("The received key is: " + k);
            System.out.println("The decrypted text is: " + cts);
            output.writeUTF("Success from receiver!");
            input.close();
            output.close();
        } catch (IOException i) {
            System.out.println(i);
        }
    }

    public static void main(String[] args) {
        Vernam_Receiver vr = new Vernam_Receiver(5431);
        vr.Work();
    }
}
```

Output (Vernam Cipher):

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1\Vernam Cipher> java .\Vernam_Sender.java
Client is connected!
Enter the message: Oak
Enter the key: son
The encrypted text is: ]oh
Success from receiver!
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1\Vernam Cipher>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1\Vernam Cipher> java .\Vernam_Receiver.java
Client is connected!

The received encrypted text is: ]oh
The received key is: son
The decrypted text is: Oak
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-1\Vernam Cipher>
```

Ex1B- Playfair and Hill Cipher

Playfair Cipher:

Code:

Sender:

```
import socket


def create_diagram(pt):
    diagram = []
    i = 0
    while i < len(pt):
        a = pt[i]
        if i + 1 < len(pt):
            b = pt[i + 1]
            if a == b:
                diagram.append(a + 'x')
                i += 1
            else:
                diagram.append(a + b)
                i += 2
        else:
            diagram.append(a + 'x')
            i += 1
    return diagram


def create_key_matrix(key):
    key = key.replace("j", "i")
    key_matrix = []
    used_chars = set()
    row = 0
    col = 0
```

```
    temp = []
    for char in key:
        if char not in used_chars and char.isalpha():
            used_chars.add(char)
            temp.append(char)
            col += 1
            if col == 5:
                key_matrix.append(temp)
                temp = []
                row += 1
                col = 0
    for i in range(ord('a'), ord('z') + 1):
        char = chr(i)
        if char == 'j':
            continue
        if char not in used_chars:
            used_chars.add(char)
            temp.append(char)
            col += 1
            if col == 5:
                key_matrix.append(temp)
                temp = []
                row += 1
                col = 0
    return key_matrix

def get_cipher(diagram, key_matrix):
    cipher_text = ""
    for pair in diagram:
        row1, col1, row2, col2 = -1, -1, -1, -1
        for r in range(5):
```

```
        for c in range(5):
            if key_matrix[r][c] == pair[0]:
                row1, col1 = r, c
            if key_matrix[r][c] == pair[1]:
                row2, col2 = r, c
    if row1 == row2:
        cipher_text += key_matrix[row1][(col1 + 1) % 5]
        cipher_text += key_matrix[row2][(col2 + 1) % 5]
    elif col1 == col2:
        cipher_text += key_matrix[(row1 + 1) % 5][col1]
        cipher_text += key_matrix[(row2 + 1) % 5][col2]
    else:
        cipher_text += key_matrix[row1][col2]
        cipher_text += key_matrix[row2][col1]
  return cipher_text

print("Playfair Cipher Sender initialized.")
pt = input("Enter the plaintext: ").lower()
if(len(pt) == 0):
  print("No plaintext provided. Exiting.")
  exit()
key = input("Enter the key: ").lower()
if(len(key) == 0):
  print("No key provided. Exiting.")
  exit()

diagram = create_diagram(pt)
diagram_text = ''.join(diagram)
print("Diagram created:", diagram_text)
key_matrix = create_key_matrix(key)
print("Key matrix created:")
```

```
for row in key_matrix:
    print(row)
cipher_text = get_cipher(diagram, key_matrix)
print("Ciphertext generated:", cipher_text)

s = socket.socket()
s.connect(('127.0.0.1', 5067))
s.send(cipher_text.encode().ljust(1024, b'\0'))
s.send(key.encode().ljust(1024, b'\0'))
print("Ciphertext and key sent to the receiver.")
s.close()
```

Receiver:

```
import socket


def create_key_matrix(key):
    key = key.replace("j", "i")
    key_matrix = []
    used_chars = set()
    row = 0
    col = 0
    temp = []
    for char in key:
        if char not in used_chars and char.isalpha():
            used_chars.add(char)
            temp.append(char)
            col += 1
            if col == 5:
                key_matrix.append(temp)
                temp = []
                row += 1
                col = 0
```

```
    for i in range(ord('a'), ord('z') + 1):
        char = chr(i)
        if char == 'j':
            continue
        if char not in used_chars:
            used_chars.add(char)
            temp.append(char)
            col += 1
            if col == 5:
                key_matrix.append(temp)
                temp = []
                row += 1
                col = 0
    return key_matrix

def get_plain_text(cipher, key_matrix):
    plain_text = ""
    for i in range(0, len(cipher), 2):
        pair = cipher[i:i+2]
        row1, col1, row2, col2 = -1, -1, -1, -1
        for r in range(5):
            for c in range(5):
                if key_matrix[r][c] == pair[0]:
                    row1, col1 = r, c
                if key_matrix[r][c] == pair[1]:
                    row2, col2 = r, c
        if row1 == row2:
            plain_text += key_matrix[row1][(col1 - 1) % 5]
            plain_text += key_matrix[row2][(col2 - 1) % 5]
        elif col1 == col2:
            plain_text += key_matrix[(row1 - 1) % 5][col1]
```

```
            plain_text += key_matrix[(row2 - 1) % 5][col2]
        else:
            plain_text += key_matrix[row1][col2]
            plain_text += key_matrix[row2][col1]
    return plain_text

server = socket.socket()
server.bind(('127.0.0.1', 5067))
server.listen(1)
print("Playfair Cipher Receiver initialized and listening for sender.")
conn, addr = server.accept()
print(f"Connection established with {addr}.")

cipher = conn.recv(1024).decode().rstrip('\0')
key = conn.recv(1024).decode().rstrip('\0')
print("Ciphertext received:", cipher)
print("Key received:", key)
key_matrix = create_key_matrix(key)
print("Plain text reconstructed:", get_plain_text(cipher, key_matrix))

conn.close()
server.close()
```

Output:

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Playfair Cipher> python .\sender.py
Playfair Cipher Sender initialized.
Enter the plaintext: hello
Enter the key: bye
Diagram created: helxlo
Key matrix created:
['b', 'y', 'e', 'a', 'c']
['d', 'f', 'g', 'h', 'i']
['k', 'l', 'm', 'n', 'o']
['p', 'q', 'r', 's', 't']
['u', 'v', 'w', 'x', 'z']
Ciphertext generated: ganvmk
Ciphertext and key sent to the receiver.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Playfair Cipher>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Playfair Cipher> python .\receiver.py
Playfair Cipher Receiver initialized and listening for sender.
Connection established with ('127.0.0.1', 57069).
Ciphertext received: ganvmk
Key received: bye
Plain text reconstructed: helxlo
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Playfair Cipher>
```

Hill Cipher:

Code:

Sender:

```
import socket

import numpy as np


def create_key_matrix(key_char_matrix):

    key_matrix = [[ord(char) - ord('a') for char in row] for row in key_char_matrix]

    return np.matrix(key_matrix, dtype=int)


def create_pt_matrix(pt, n):

    pt_matrix = []

    for i in range(0, len(pt), n):

        row = [ord(char) - ord('a') for char in pt[i:i+n]]

        pt_matrix.append(row)

    return np.matrix(pt_matrix, dtype=int)


def get_cipher_text(ct_matrix):

    ct = ''

    for row in ct_matrix:

        for num in row.tolist()[0]:

            ct += chr(num + ord('a'))

    return ct


def get_key_text(key_char_matrix):

    key = ''

    for row in key_char_matrix:

        for char in row:

            key += char

    return key
```

```
print("Hill Cipher Sender initialized.")

n = int(input("Enter the size of the key matrix (n x n): "))

pt = input(f"Enter the plaintext: ").lower()

key_char_matrix = [list(input(f"Enter row {i+1} of the key matrix (space-separated integers and length must be {n}): ").lower()) for i in range(n)]

key_matrix = create_key_matrix(key_char_matrix)

key_text = get_key_text(key_char_matrix)

print("Key Matrix:")

for row in key_matrix:

    print(row)

print(f"Key Text: {key_text}")


pt_matrix = create_pt_matrix(pt, n)

print("Plaintext Matrix:")

for row in pt_matrix:

    print(row)


ct_matrix = (pt_matrix @ key_matrix) % 26

print("Ciphertext Matrix:")

for row in ct_matrix:

    print(row)


ct = get_cipher_text(ct_matrix)

print(f"Ciphertext: {ct}")


s = socket.socket()

s.connect(('127.0.0.1', 5069))

s.send(ct.encode().ljust(1024, b'\0'))

s.send(str(n).encode().ljust(1024, b'\0'))

s.send(key_text.encode().ljust(1024, b'\0'))

print("Ciphertext, key size, and key text sent to receiver.")

s.close()
```

Receiver:

```
import socket

import numpy as np


def create_key_matrix(key_text, n):

    key_matrix = []

    index = 0

    for i in range(n):

        row = []

        for j in range(n):

            row.append(ord(key_text[index]) - ord('a'))

            index += 1

        key_matrix.append(row)

    return np.matrix(key_matrix, dtype=int)


def create_ct_matrix(ct, n):

    ct_matrix = []

    for i in range(0, len(ct), n):

        row = [ord(char) - ord('a') for char in ct[i:i+n]]

        ct_matrix.append(row)

    return np.matrix(ct_matrix, dtype=int)


def get_key_matrix_adjugate(key_matrix):

    n = key_matrix.shape[0]

    adjugate = np.zeros((n, n), dtype=int)

    for i in range(n):

        for j in range(n):

            minor = np.delete(np.delete(key_matrix, i, axis=0), j, axis=1)

            cofactor = int(round(np.linalg.det(minor))) * ((-1) ** (i + j))

            adjugate[j][i] = cofactor % 26

    return np.asmatrix(adjugate)
```

```
def get_plain_text(pt_matrix):
    pt = ''
    for row in pt_matrix:
        for num in row.tolist()[0]:
            pt += chr(num + ord('a'))
    return pt

server = socket.socket()
server.bind(('127.0.0.1', 5069))
server.listen(1)
print("Hill Cipher Receiver initialized and listening for sender.")

conn, addr = server.accept()
print(f"Connection established with sender at {addr}.")
ct = conn.recv(1024).decode().rstrip('\0')
n = int(conn.recv(1024).decode().rstrip('\0'))
key_text = conn.recv(1024).decode().rstrip('\0')

print(f"Received Ciphertext: {ct}")
print(f"Received Key Size: {n}")
print(f"Received Key Text: {key_text}")

key_matrix = create_key_matrix(key_text, n)
print("Key Matrix:")
for row in key_matrix:
    print(row)

ct_matrix = create_ct_matrix(ct, n)
print("Ciphertext Matrix:")
for row in ct_matrix:
```

```
    print(row)

det = int(round(np.linalg.det(key_matrix))) % 26
det_inverse = pow(det, -1, 26)
print(f"Determinant: {det}")
print(f"Determinant Inverse: {det_inverse}")

key_matrix_inverse = (det_inverse * get_key_matrix_adjugate(key_matrix)) % 26
print("Inverse Key Matrix:")
for row in key_matrix_inverse:
    print(row)

pt_matrix = (ct_matrix @ key_matrix_inverse) % 26
print("Plaintext Matrix:")
for row in pt_matrix:
    print(row)

pt = get_plain_text(pt_matrix)
print(f"Decrypted Plaintext: {pt}")

conn.close()
server.close()
```

Output:

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Hill Cipher> python .\sender.py
Hill Cipher Sender initialized.
Enter the size of the key matrix (n x n): 3
Enter the plaintext: paymoremoney
Enter row 1 of the key matrix (space-separated integers and length must be 3): rrf
Enter row 2 of the key matrix (space-separated integers and length must be 3): vsv
Enter row 3 of the key matrix (space-separated integers and length must be 3): cct
Key Matrix:
[[17 17  5]]
[[21 18 21]]
[[ 2  2 19]]
Key Text: rrfvsvcct
Plaintext Matrix:
[[15  0 24]]
[[12 14 17]]
[[ 4 12 14]]
[[13  4 24]]
Ciphertext Matrix:
[[17 17 11]]
[[12 22  1]]
[[10  0 18]]
[[15  3  7]]
Ciphertext: rrlmwbkaspdh
Ciphertext, key size, and key text sent to receiver.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Hill Cipher>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Hill Cipher> python .\receiver.py
Hill Cipher Receiver initialized and listening for sender.
Connection established with sender at ('127.0.0.1', 49701).
Received Ciphertext: rrlmwbkaspdh
Received Key Size: 3
Received Key Text: rrfvsvcct
Key Matrix:
[[17 17  5]]
[[21 18 21]]
[[ 2  2 19]]
Ciphertext Matrix:
[[17 17 11]]
[[12 22  1]]
[[10  0 18]]
[[15  3  7]]
Determinant: 23
Determinant Inverse: 17
Inverse Key Matrix:
[[ 4  9 15]]
[[15 17  6]]
[[24  0 17]]
Plaintext Matrix:
[[15  0 24]]
[[12 14 17]]
[[ 4 12 14]]
[[13  4 24]]
Decrypted Plaintext: paymoremoney
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Hill Cipher>
```

Ex 1C – Transposition Cryptographic Algorithms

Rail Fence Cipher:

Code:

Sender:

```
import socket


print("Rail Fence Cipher Sender Initialized.")

pt = input("Enter the text to be encoded: ")

d = int(input("Enter the depth: "))


ar = [[None for i in range(len(pt))] for j in range(d)]

row, col = 0,0

direction = 1


for i in pt:

    ar[row][col] = i

    col += 1

    row += direction

    if row == 0 or row == d-1:

        direction *= -1


print("\nThe intermediate matrix is:")

for i in ar:

    for j in i:

        if j == None:

            print("-", end=" ")

        else:

            print(j, end=" ")
```

```
    print()
print()

ct = ""
for i in ar:
    for j in i:
        if j != None:
            ct += j

print("The Cipher Text is: " +  ct)

s = socket.socket()
s.connect(('127.0.0.1', 12345))
s.send(ct.encode().ljust(1024, b'\0'))
s.send(str(d).encode().ljust(1024, b'\0'))
print("Cipher text and the depth are sent successfully.")
s.close()
```

Receiver:

```
import socket

server = socket.socket()
server.bind(('127.0.0.1', 12345))
server.listen(1)
print("Rail Fence Cipher Receiver initialized and listening for sender.")

conn, addr = server.accept()
print(f"Connection established with sender at {addr}.")
ct = conn.recv(1024).decode().rstrip('\0')
d = int(conn.recv(1024).decode().rstrip('\0'))
```

```
print(f"Received Cipher Text is: {ct}.")
print(f"Received depth is: {d}.")

row, col = 0,0
direction = 1
ar1 = [[None for i in range(len(ct))] for j in range(d)]
for i in ct:
    ar1[row][col] = '*'
    col += 1
    row += direction
    if row == 0 or row == d-1:
        direction *= -1

index = 0
for i in range(d):
    for j in range(len(ct)):
        if index < len(ct) and ar1[i][j] == '*':
            ar1[i][j] = ct[index]
            index += 1

print("The intermediate matrix is:")
for i in ar1:
    for j in i:
        if j == None:
            print("-", end=" ")
        else:
            print(j, end=" ")
    print()
print()

row, col = 0,0
```

```
direction = 1
pt1 = ""
while len(pt1) < len(ct):
    pt1 += ar1[row][col]
    col += 1
    row += direction
    if row == 0 or row == d-1:
        direction *= -1
print("The plaintext is: " + pt1)

conn.close()
server.close()
```

Output:

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-3\Rail Fence> python .\sender.py
Rail Fence Cipher Sender Initialized.
Enter the text to be encoded: meetmeafterthetogaparty
Enter the depth: 3

The intermediate matrix is:
m - - - m - - - t - - - h - - - g - - - r - -
- e - t - e - f - e - t - e - o - a - a - t -
- - e - - - a - - - r - - - t - - - p - - - y

The Cipher Text is: mmthgretefeteoaateartpy
Cipher text and the depth are sent successfully.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-3\Rail Fence>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-3\Rail Fence> python .\receiver.py
Rail Fence Cipher Receiver initialized and listening for sender.
Connection established with sender at ('127.0.0.1', 59870).
Received Cipher Text is: mmthgretefeteoaateartpy.
Received depth is: 3.
The intermediate matrix is:
m - - - m - - - t - - - h - - - g - - - r - -
- e - t - e - f - e - t - e - o - a - a - t -
- - e - - - a - - - r - - - t - - - p - - - y

The plaintext is: meetmeafterthetogaparty
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-3\Rail Fence>
```

Row Column Transposition Cipher:

Code:

Sender:

```
import pickle

import socket


print("Row Column Cipher Sender Initialized.")

pt = input("Enter the plaintext: ")

pt_len = len(pt)

key = input("Enter the permutation: ").split(" ")

key = [int(i) for i in key]

key_data = pickle.dumps(key)


if max(key) != len(key) or min(key) != 1:

    print("The given permutation is invalid.")

    exit(1)


if len(pt) % len(key) != 0:

    pad = len(key) - (pt_len % len(key))

    pt += '*'*pad


mat = [[None for i in range(len(key))] for j in range(len(pt)//len(key))]

index = 0

for i in range(len(pt)//len(key)):

    for j in range(len(key)):

        if index < len(pt):

            mat[i][j] = pt[index]

            index += 1

print("\nThe intermediate matrix is:")

for i in mat:

    for j in i:
```

```
        print(j, end=" ")
    print()
print()

d = dict()
for i in range(len(key)):
    d[key[i]] = ""
    for j in range(len(pt)//len(key)):
        d[key[i]] += mat[j][i]

ct = ""
for i in sorted(d.keys()):
    ct += d[i]
print(f"The Cipher Text is: \"{ct}\".")

s = socket.socket()
s.connect(('127.0.0.1', 5096))
s.send(ct.encode().ljust(1024, b'\0'))
s.send(key_data.ljust(1024, b'\0'))
s.send(str(pt_len).encode().ljust(1024, b'\0'))
print("The Ciphertext, permutation and the original plain text length have been sent successfully.")
s.close()
```

Receiver:

```
import socket
import pickle

server = socket.socket()
server.bind(('127.0.0.1', 5096))
server.listen(1)
print("Row Column Cipher Receiver initialized and listening for sender.")
```

```
conn, addr = server.accept()
print(f"Connection established with sender at {addr}.")
ct = conn.recv(1024).decode().rstrip('\0')
print(f"The Cipher Text is: {ct}.")
key = pickle.loads(conn.recv(1024).rstrip(b'\0'))
pt_len = int(conn.recv(1024).decode().rstrip('\0'))
print(f"The permutation is: {key}")

conn.close()
server.close()

d = dict()
gap = len(ct)//len(key)
for i in range(1, len(key)+1):
    d[i] = ct[(i-1)*gap : i*gap]

mat = [[None for i in range(len(key))] for j in range(gap)]
for i in range(len(key)):
    block = d[key[i]]
    for j in range(gap):
        mat[j][i] = block[j]
print("\nThe intermediate matrix is:")
pt = ""
for i in mat:
    for j in i:
        print(j, end=" ")
        pt += j
    print()
print()
```

```
print(f"The Plain Text with padding is: \"{pt}\".")

pt = pt[:pt_len]

print(f"The Plain Text without padding is: \"{pt}\".")
```

Output:

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-3\Row Column> python .\sender.py
Row Column Cipher Sender Initialized.
Enter the plaintext: attackpostponeduntiltwoam
Enter the permutation: 4 3 1 2 5 6 7

The intermediate matrix is:
a t t a c k p
o s t p o n e
d u n t i l t
w o a m * * *

The Cipher Text is: "ttnaaptmtsuoaodwcoi*knl*pet*".
The Ciphertext, permutation and the original plain text length have been sent successfully.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-3\Row Column>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-3\Row Column> python .\receiver.py
Row Column Cipher Receiver initialized and listening for sender.
Connection established with sender at ('127.0.0.1', 53382).
The Cipher Text is: ttnaaptmtsuoaodwcoi*knl*pet*.
The permutation is: [4, 3, 1, 2, 5, 6, 7]

The intermediate matrix is:
a t t a c k p
o s t p o n e
d u n t i l t
w o a m * * *

The Plain Text with padding is: "attackpostponeduntiltwoam***".
The Plain Text without padding is: "attackpostponeduntiltwoam".
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-3\Row Column>
```